Based on the official documentation, the core Soccermatics course is primarily designed to run locally. The "Getting Started" materials instruct students to install a local Python environment using Anaconda and IDEs like Spyder to process the data. However, in a modern professional environment, relying solely on local compute is insufficient for massive tracking datasets. Transitioning this to a cloud environment like AWS is a perfect exercise to elevate your infrastructure to an enterprise-grade standard.

Here is a specific, step-by-step implementation plan based on the architectural diagram you provided. You can hand this directly to an AI coding assistant (like Claude Code) to generate the Infrastructure as Code (Terraform or AWS CloudFormation) and the necessary Python scripts.

Phase 1: Infrastructure Provisioning (IaC)

Ask the coding assistant to generate code to provision the following AWS services, ensuring all necessary IAM roles and VPC networking (security groups allowing traffic between services) are configured:

- **Amazon S3:** Create two buckets: a raw-zone for initial data ingestion and a processed-zone for transformed data.
- **Amazon MWAA (Managed Workflows for Apache Airflow):** Provision an MWAA environment to replace the manual EC2 Airflow setup depicted in the image. This will act as the master orchestrator.
- **AWS Glue:** Provision a Glue Data Catalog and an IAM role allowing Glue to read/write to your S3 buckets.
- **Amazon Redshift:** Provision a Redshift cluster (or Redshift Serverless) to act as the primary data warehouse.
- **Amazon RDS for PostgreSQL:** Provision a PostgreSQL instance to serve as the final OLTP database for your downstream web application (like Streamlit).

Phase 2: Data Ingestion (Airflow DAGs)

Have the assistant write an Airflow DAG with Python Operators to extract data from the open-source providers and load it into the S3 raw-zone.

- **StatsBomb Data:** Use the statsbombpy Python library or standard requests to pull open event and 360-tracking JSON data from their public API/GitHub.

- **Metrica Sports Data:** Fetch the sample optical tracking (CSV) and event data directly from their sample-data GitHub repository.

- **Wyscout Data:** Ingest the public release of the Wyscout event stream dataset (JSON).

Phase 3: Serverless ETL (AWS Glue)

Instruct the assistant to write AWS Glue PySpark scripts to perform the "EXTRACT" and transformation steps shown in your diagram:

- Read the raw JSON and CSV files from the S3 raw-zone.
- Flatten the highly nested event structures (e.g., unpacking arrays containing passing coordinates and pressure events).
- Cast data types and write the cleaned data to the S3 processed-zone in Apache Parquet format for optimized querying.

Phase 4: Warehousing and Modeling (Redshift & dbt)

Ask the assistant to write the Airflow tasks and dbt configurations required to load and model the data:

- **The LOAD Step:** Write an Airflow PostgresOperator (connecting to Redshift) to execute the COPY command. This command loads the Parquet files directly from the S3 processed-zone into staging tables in Redshift.
- **The dbt Transformations:** Configure a dbt-redshift project. Have the assistant generate sample dbt SQL models that take the raw staging tables and build aggregated analytical tables (e.g., calculating expected goals, mapping pass networks, or calculating player distance covered).

Phase 5: Reverse ETL to OLTP (PostgreSQL)

To complete the pipeline exactly as depicted in the image, you must move the modeled data out of the OLAP warehouse and into the OLTP database for low-latency application serving:

- Have the assistant write an Airflow task that executes a Redshift UNLOAD command, pushing the final dbt models back to S3 as CSV files.
- Write a subsequent Airflow task that connects to the Amazon RDS PostgreSQL instance and uses the s3_import extension (or a standard Postgres COPY) to ingest those files into the operational database.